AI Banking Assistant - Use Cases

Human-in-the-Loop Escalation Flow

This document describes two primary use cases for the AI Banking Assistant, demonstrating how the system decides whether to handle a request automatically (AUTO) or escalate it to a human support agent (ESCALATE).

The 'status' field in every response drives the human-in-the-loop decision:
- AUTO = AI handled the request successfully, no human needed
- ESCALATE = AI cannot complete the request, route to human agent

Use Case 1: Valid Contract ID (Happy Path)

Scenario

A banking customer sends a message asking about their loan contract and provides a valid contract_id (C123) that exists in the DynamoDB Contracts table.

Request

```
POST /chat
Content-Type: application/json

{
  "message": "Can you show me the details of my loan contract?",
  "contract_id": "C123"
}
```

Processing Flow

1. Customer sends message with contract_id = 'C123'
2. API validates request (message present, under 1000 chars)
3. DynamoDB Client queries Contracts table for 'C123'
4. Contract FOUND - record returned (amount, interest_rate, duration)
5. Bedrock Client generates 3-bullet-point summary of the contract
6. API returns response with summary and status AUTO

Expected Response (HTTP 200)

```
{
  "message": "Contract C123 found and summarized successfully.",
  "contract_summary": "- Contract amount: $50,000\n- Interest rate: 5%\n- Duration: 5 years",
  "status": "AUTO"
}
```

STATUS: AUTO

Human-in-the-Loop Decision

NO ESCALATION NEEDED. The AI successfully:
1. Found the contract in DynamoDB
2. Generated a clear summary via Bedrock LLM
3. Returned the result to the customer

The frontend displays the summary directly to the customer. No support ticket is created. No human agent is involved.

AI Banking Assistant - Use Cases

Human-in-the-Loop Escalation Flow

Use Case 2: Invalid Contract ID (Human-in-the-Loop Escalation)

Scenario

A banking customer sends a message with a contract_id that does NOT exist in the DynamoDB Contracts table. The AI cannot find the contract, so it escalates the request to a human support agent who can manually assist.

Request

```
POST /chat
Content-Type: application/json

{
  "message": "Please check my loan status",
  "contract_id": "INVALID_CONTRACT_999"
}
```

Processing Flow

1. Customer sends message with contract_id = 'INVALID_CONTRACT_999'
2. API validates request (message present, under 1000 chars)
3. DynamoDB Client queries Contracts table for 'INVALID_CONTRACT_999'
4. Contract NOT FOUND - DynamoDB returns None
5. API triggers escalation - human support needed
6. API returns response with status ESCALATE

Expected Response (HTTP 200)

```
{
  "message": "Contract not found, escalating to support",
  "contract_summary": null,
  "status": "ESCALATE"
}
```

STATUS: ESCALATE

Human-in-the-Loop Decision

ESCALATION TRIGGERED! The system cannot resolve this request automatically.

What happens next:
1. Frontend detects status = 'ESCALATE'
2. System creates a support ticket with the customer's context
3. Human agent receives the ticket with:
   - Customer message: 'Please check my loan status'
   - Attempted contract_id: 'INVALID_CONTRACT_999'
4. Human agent investigates:
   - Is the contract_id a typo?
   - Was the contract closed or transferred?
   - Does the customer need help finding their correct contract ID?
5. Human agent responds directly to the customer

AI Banking Assistant - Use Cases

Human-in-the-Loop Escalation Flow

Why Escalation Matters

Without human-in-the-loop, the customer would be stuck. The AI correctly identifies that it cannot help (contract doesn't exist) and hands off to a human who has access to additional systems and context to resolve the issue.

This prevents:
- Frustrated customers stuck in an AI loop
- Incorrect information being provided
- Loss of customer trust when the AI pretends to have answers it doesn't have

AI Banking Assistant - Use Cases

Human-in-the-Loop Escalation Flow

Comparison: AUTO vs ESCALATE

Aspect	Use Case 1 (Valid)	Use Case 2 (Invalid)
Contract ID	C123 (exists)	INVALID_CONTRACT_999
DynamoDB Result	Record found	None (not found)
Bedrock Called?	Yes - generates summary	No - never reached
Response Status	AUTO	ESCALATE
HTTP Code	200	200
contract_summary	Populated with summary	null
Human Needed?	No	Yes - support agent
Customer Experience	Gets instant answer	Routed to live support

Visual Flow Summary

Use Case 1: Valid Contract (AUTO)

```
Customer ---> POST /chat (contract_id=C123)
   |
   v
FastAPI ---> DynamoDB (query contract_id=C123)
   |
   v         FOUND! Returns ContractRecord
   |
   v
FastAPI ---> Bedrock LLM (generate summary)
   |
   v         Summary generated successfully
   |
   v
Customer <--- {status: AUTO, contract_summary: '...'}

Result: Customer sees contract summary. Done.
```

Use Case 2: Invalid Contract (ESCALATE)

```
Customer ---> POST /chat (contract_id=INVALID_CONTRACT_999)
   |
   v
FastAPI ---> DynamoDB (query contract_id=INVALID_CONTRACT_999)
   |
   v         NOT FOUND! Returns None
   |
   v
FastAPI ---> ESCALATION TRIGGERED
   |
   v
Customer <--- {status: ESCALATE, message: 'Contract not found...'}
```

AI Banking Assistant - Use Cases

Human-in-the-Loop Escalation Flow

```
   |
   v
Support System ---> Creates ticket ---> Human Agent picks up
   |
   v
Human Agent ---> Contacts customer ---> Resolves issue
```

AI Banking Assistant - Use Cases

Human-in-the-Loop Escalation Flow

Quick Test Commands

Start the server:

```
uvicorn main:app --reload --port 8000
```

Use Case 1 - Valid Contract:

```
curl -s -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Show my contract", "contract_id": "C123"}' | python3 -m json.tool
```

Use Case 2 - Invalid Contract:

```
curl -s -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Check loan status", "contract_id": "INVALID999"}' | python3 -m json.tool
```

Run automated demo script:

```
python demo_use_cases.py
```